ALLIANCE UNIVERSITY · BCA (AI/ML) · DBMS ASSIGNMENT SERIES

08

ASSIGNMENT 8

Basic SQL Queries and Aggregate Functions

Online Food Delivery System

SUBMITTED BY
Vimal Kumar Rao

PROGRAMME
BCA (AI/ML)

INSTITUTION
Alliance University

SUBMISSION DATE
24 October 2026

IN THIS ASSIGNMENT

01 Filter rows

02 Sort results

03 Group & aggregate

04 Explain output

A consistent relational design carried forward from Assignments 1 and 2.

1 Objective and Brief Coverage

This assignment queries the Online Food Delivery database with the core SQL clauses: **WHERE**, **ORDER BY**, **GROUP BY**, **HAVING** and the five aggregate functions **COUNT, SUM, AVG, MIN, MAX**. Every query below is shown with its SQL, the result it returns on the Assignment 7 sample data, and a short explanation. Two bonus queries extend the required set.

12	21	Rs 4,225	10
orders in the dataset	order lines	non-cancelled line value	queries (8 + 2 bonus)

	Brief requirement	Where it is covered
✓	**Filter with WHERE**	Q1, Q9
✓	**Sort with ORDER BY**	Q1, Q2 (also used in every other query)
✓	**Group with GROUP BY**	Q3, Q4, Q6, Q7, Q8, Q9
✓	**Filter groups with HAVING**	Q4, Q9
✓	**COUNT, SUM, AVG, MIN, MAX**	Q5 (all five together); COUNT / SUM again in Q3 to Q9
✓	**SQL, output and explanation**	Every query block: SQL, OUTPUT table, "Reading the result"

How MySQL reads a query

The order you *write* the clauses is not the order MySQL *evaluates* them. Knowing the evaluation order explains why WHERE cannot see an aggregate, while HAVING can, and why ORDER BY can use a column alias.

1	2	3	4	5	6	7
FROM / JOIN build the working rows	**WHERE** drop rows	**GROUP BY** form groups	**HAVING** drop groups	**SELECT** compute columns, aliases	**ORDER BY** sort	**LIMIT** trim

Dataset used

Results come from the 84 rows loaded by **Assignment 7** (7 customers, 5 restaurants, 15 menu items, 12 orders, 21 order lines, 12 payments). All amounts are fictional and in Indian rupees.

2 Required Queries (Q1 to Q8)

Q1 WHERE: delivered orders

WHERE ORDER BY

Filters order headers to those with status **Delivered** and lists the newest first. **WHERE** removes rows *before* anything is grouped or returned.

```
SELECT Order_ID, Order_Date, Order_Status
FROM   Orders
WHERE  Order_Status = 'Delivered'
ORDER BY Order_Date DESC;
```

OUTPUT 6 rows

Order_ID	Order_Date	Order_Status
8	2026-10-24 08:45:00	Delivered
4	2026-10-22 20:00:00	Delivered
3	2026-10-22 13:20:00	Delivered
2	2026-10-21 19:05:00	Delivered
1	2026-10-21 12:15:00	Delivered
11	2026-10-20 20:00:00	Delivered

Reading the result

Six of the twelve orders are Delivered. Order 11 (20 Oct) is the oldest and order 8 (24 Oct, 08:45) the newest, because the sort is on the date, not the ID.

Q2 ORDER BY and LIMIT: most expensive menu items

ORDER BY | LIMIT | tie-breaker

Ranks menu items by current price, highest first. Item_Name is a tie-breaker so equal prices always come out in the same order; **LIMIT 5** keeps the top five.

```
SELECT Item_ID, Item_Name, Price
FROM   Menu_Item
ORDER BY Price DESC, Item_Name ASC
LIMIT 5;
```

OUTPUT 5 rows

Item_ID	Item_Name	Price
8	Farmhouse Pizza	329
7	Margherita Pizza	249
1	Chicken Biryani	220
14	Chilli Paneer	220
2	Paneer Biryani	180

Reading the result

Farmhouse Pizza (329) leads. Chicken Biryani and Chilli Paneer tie at 220, and the alphabetical tie-breaker puts Chicken Biryani first.

Q3 GROUP BY: orders per restaurant

LEFT JOIN | GROUP BY | COUNT

Counts order headers for every restaurant. A **LEFT JOIN** keeps restaurants that have zero orders, so none disappear from the report.

```
SELECT r.Restaurant_Name,
       COUNT(o.Order_ID) AS Total_Orders
FROM   Restaurant r
LEFT JOIN Orders o ON o.Restaurant_ID = r.Restaurant_ID
GROUP BY r.Restaurant_ID, r.Restaurant_Name
ORDER BY Total_Orders DESC, r.Restaurant_Name ASC;
```

OUTPUT 5 rows

Restaurant_Name	Total_Orders
Spice Route Kitchen	3
Urban Pizza Co.	3
Dosa Junction	2
Green Bowl Cafe	2
Wok Express	2

Reading the result

Spice Route Kitchen and Urban Pizza Co. are the busiest with 3 orders each (this count includes the cancelled order 12). Counting o.Order_ID rather than * would correctly give 0 for a restaurant with no orders.

Q4 HAVING: repeat customers

JOIN GROUP BY HAVING

Groups orders by customer and keeps only groups with at least two orders. **HAVING** filters after aggregation; WHERE could not do this because COUNT does not exist yet at that stage.

```sql
SELECT c.Name, COUNT(o.Order_ID) AS Order_Count
FROM   Customer c
JOIN   Orders o ON o.Customer_ID = c.Customer_ID
GROUP BY c.Customer_ID, c.Name
HAVING COUNT(o.Order_ID) >= 2
ORDER BY Order_Count DESC, c.Name ASC;
```

OUTPUT 5 rows

Name	Order_Count
Arjun Reddy	2
Asha Menon	2
Meera Iyer	2
Priya Sharma	2
Rahul Nair	2

Reading the result

Five of the seven customers ordered twice. Kabir Das and Nisha Rao have a single order each, so HAVING removes them.

Q5 Aggregate functions: COUNT, SUM, AVG, MIN, MAX

COUNT SUM AVG MIN MAX

Summarises every non-cancelled order line in one row: number of lines, total value, average / lowest / highest unit price.

```sql
SELECT COUNT(*)                          AS Item_Lines,
       SUM(oi.Quantity * oi.Unit_Price)  AS Line_Value_Total,
       ROUND(AVG(oi.Unit_Price), 2)      AS Average_Unit_Price,
       MIN(oi.Unit_Price)                AS Lowest_Unit_Price,
       MAX(oi.Unit_Price)                AS Highest_Unit_Price
FROM   Order_Item oi
JOIN   Orders o ON o.Order_ID = oi.Order_ID
WHERE  o.Order_Status <> 'Cancelled';
```

OUTPUT 1 row

Item_Lines	Line_Value_Total	Average_Unit_Price	Lowest_Unit_Price	Highest_Unit_Price
20	4225	162.30	40	329

Reading the result

20 lines (the 21st belongs to cancelled order 12) are worth Rs 4,225 in total. The average is a *simple* average of unit prices, not weighted by quantity, so it is not the average spend per item sold.

Q6 GROUP BY and SUM: subtotal by order status

JOIN | GROUP BY | COUNT DISTINCT | SUM

Shows how many orders and how much line-item value sit in each status. COUNT(DISTINCT Order_ID) is needed because joining to order lines repeats each order once per line.

```
SELECT o.Order_Status,
       COUNT(DISTINCT o.Order_ID)        AS Orders,
       SUM(oi.Quantity * oi.Unit_Price)  AS Line_Subtotal
FROM   Orders o
JOIN   Order_Item oi ON oi.Order_ID = o.Order_ID
GROUP BY o.Order_Status
ORDER BY Orders DESC, o.Order_Status ASC;
```

OUTPUT 6 rows

Order_Status	Orders	Line_Subtotal
Delivered	6	2196
Placed	2	729
Cancelled	1	150
Confirmed	1	420
Out for Delivery	1	480
Preparing	1	400

Reading the result

Delivered orders account for Rs 2,196. Cancelled shows Rs 150 but is **not** sales. Plain COUNT(*) here would count order lines (21), not orders (12).

Q7 GROUP BY date: daily subtotal

DATE() GROUP BY COUNT DISTINCT SUM

Aggregates non-cancelled order lines by calendar day to show daily demand. DATE() strips the time so all orders on a day fall into one group.

```
SELECT DATE(o.Order_Date)              AS Order_Day,
       COUNT(DISTINCT o.Order_ID)        AS Orders,
       SUM(oi.Quantity * oi.Unit_Price)  AS Line_Subtotal
FROM   Orders o
JOIN   Order_Item oi ON oi.Order_ID = o.Order_ID
WHERE  o.Order_Status <> 'Cancelled'
GROUP BY DATE(o.Order_Date)
ORDER BY Order_Day ASC;
```

OUTPUT 5 rows

Order_Day	Orders	Line_Subtotal
2026-10-20	1	498
2026-10-21	2	890
2026-10-22	2	628
2026-10-23	3	1209
2026-10-24	3	1000

Reading the result

23 October is the peak day at Rs 1,209 from 3 orders. 20 October shows 1 order because the other order that day (12) was cancelled and filtered out.

Q8 GROUP BY: payment status distribution

GROUP BY COUNT

Counts payment records per status so that Pending and Failed rows, which need follow-up, are easy to spot.

```
SELECT Payment_Status, COUNT(*) AS Payment_Records
FROM   Payment
GROUP BY Payment_Status
ORDER BY Payment_Records DESC, Payment_Status ASC;
```

OUTPUT 4 rows

Payment_Status	Payment_Records
Paid	6
Pending	4
Failed	1
Refunded	1

Reading the result

Half the payments (6 of 12) are Paid. 4 are Pending and 1 has Failed, so 5 orders need payment follow-up (the same five rows returned by Assignment 9, Q6).

3 Bonus Queries

Two extra queries show HAVING on a calculated total and pattern matching with LIKE.

Q9 BONUS: HAVING on an aggregate, orders above Rs 400

GROUP BY SUM HAVING

Finds individual orders whose line total exceeds Rs 400, ignoring cancelled orders. This combines WHERE (row filter) with HAVING (group filter) in one statement.

```
SELECT o.Order_ID,
       SUM(oi.Quantity * oi.Unit_Price) AS Order_Total
FROM   Orders o
JOIN   Order_Item oi ON oi.Order_ID = o.Order_ID
WHERE  o.Order_Status <> 'Cancelled'
GROUP BY o.Order_ID
HAVING SUM(oi.Quantity * oi.Unit_Price) > 400
ORDER BY Order_Total DESC;
```

OUTPUT 4 rows

Order_ID	Order_Total
1	590
11	498
5	480
9	420

Reading the result

Four orders clear Rs 400. Order 1 (Rs 590) is the largest. WHERE ran first and removed order 12; HAVING then removed every order at or under 400.

Q10 BONUS: LIKE pattern search

LIKE ORDER BY

Pattern matching with **LIKE**: finds every menu item whose name contains the word Paneer, cheapest first.

```
SELECT Item_ID, Item_Name, Price
FROM   Menu_Item
WHERE  Item_Name LIKE '%Paneer%'
ORDER BY Price ASC;
```

OUTPUT 3 rows

Item_ID	Item_Name	Price
5	Paneer Wrap	130
2	Paneer Biryani	180
14	Chilli Paneer	220

Reading the result

% matches any run of characters, so the pattern finds Paneer at the start (Paneer Wrap, Paneer Biryani) or the end (Chilli Paneer) of a name.

4 Visual Summary

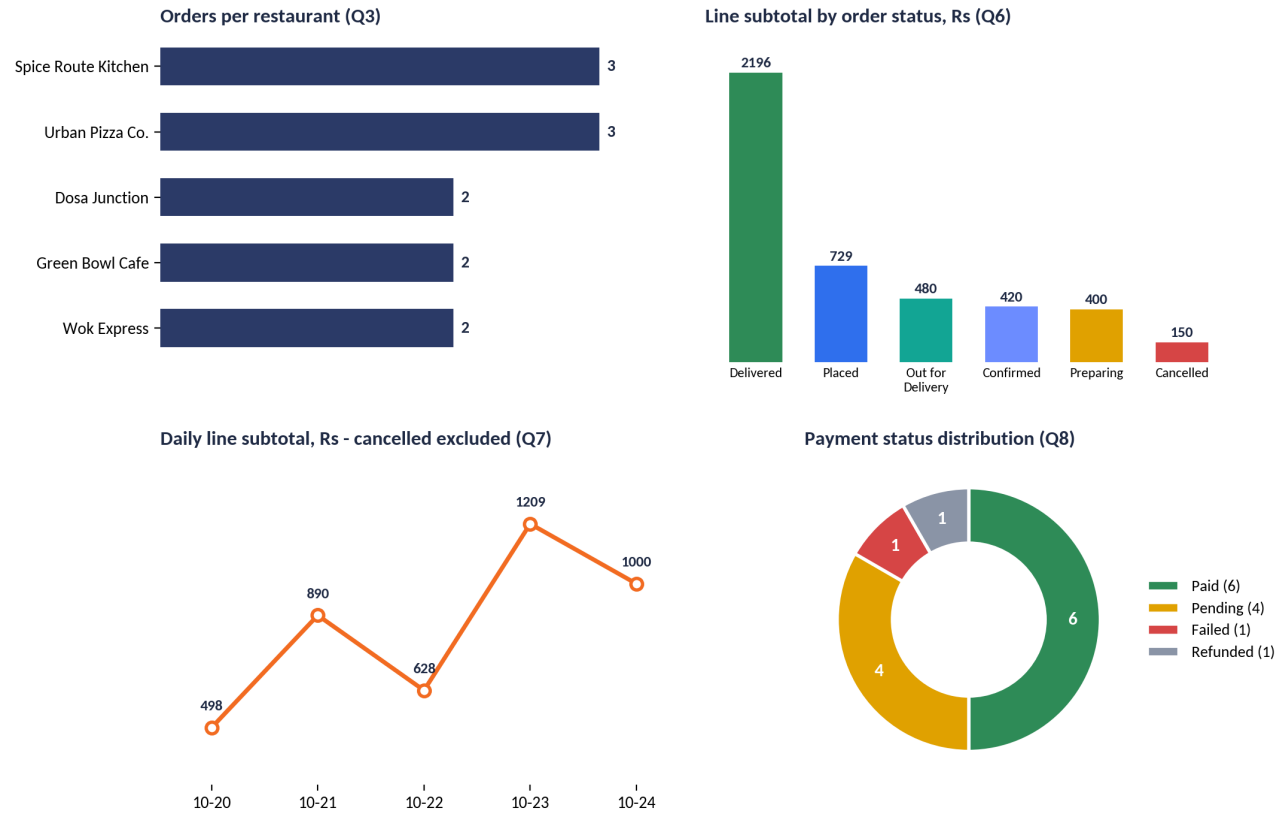


Figure 1. Charts drawn from the results of Q3, Q6, Q7 and Q8.

5 Interpretation and Pitfalls

- **Row questions vs group questions.** WHERE and ORDER BY answer "which rows?" (delivered orders, priciest items). GROUP BY with COUNT / SUM answers "how many / how much per ...?"; HAVING then filters those groups.
- **Join fan-out.** Joining Orders to Order_Item repeats each order once per line. Use COUNT(DISTINCT Order_ID) when counting orders after such a join (Q6, Q7).
- **Historical prices.** Line subtotals use Order_Item.Unit_Price, the price at order time, never today's Menu_Item.Price.
- **Business definitions matter.** Cancelled orders are excluded where stated. Whether Pending or Refunded orders count as revenue is a business decision these queries do not make.
- **Aggregate semantics.** AVG(Unit_Price) in Q5 is unweighted; a quantity-weighted average would be SUM(Quantity*Unit_Price)/SUM(Quantity).

6 Execution and Verification

- Run Assignments 5, 6 and 7 first (or **Complete_Online_Food_Delivery_Setup.sql**), then open **Assignment_8_Basic_SQL_Queries.sql** in MySQL Workbench and run each SELECT separately.
- Compare each result grid with the OUTPUT tables above.
- The OUTPUT tables were produced by executing exactly the SQL printed here against the exact INSERT rows from the setup script, using a local SQLite validation harness (no MySQL server was available while preparing this document). The statements use standard syntax that MySQL 8 accepts; take Workbench screenshots from your own server as final evidence.